Real-time Target Speaker Extraction

Grant Booysen

25849646@sun.ac.za

Thesis presented in partial fulfilment of the requirements for the degree of

MSc in Machine Learning & Artificial Intelligence

in the Faculty of Science at Stellenbosch University

Supervisors: DJ Swanevelder and David Botha (SmartOperator)

November 2026

Declaration

By submitting this thesis electronically, I declare that the entirety of the work contained therein is my own, original work, that I am the sole author thereof (save to the extent explicitly otherwise stated), that reproduction and publication thereof by Stellenbosch University will not infringe any third party rights and that I have not previously in its entirety or in part submitted it for obtaining any qualification.

Grant Booysen	**November 2026**
Signature	Date

Abstract

Acknowledgements

Contents

Nomenclature

Abbreviations

AMI	Augmented multi-party interaction
ASR	Automatic speech recognition
BAK	Background intrusiveness (DNSMOS P.835 sub-score)
BSRNN	Band-split recurrent neural network
DNSMOS	Deep noise suppression mean opinion score
EQ	Equalisation
FR	Fabrication rate
GLU	Gated linear unit
ICR	Interferer content rate
IHM	Individual headset microphone
ITU-T	ITU Telecommunication Standardization Sector
LCF-WER	Live-model content fidelity word error rate
LSTM	Long short-term memory
LUFS	Loudness units relative to full scale
MLP	Multilayer perceptron
NLTK	Natural Language Toolkit
OVRL	Overall quality (DNSMOS P.835 sub-score)
REAL-TSE	Real-world target speaker extraction (challenge)
RIR	Room impulse response
SAR	Signal-to-artefacts ratio
SDR	Signal-to-distortion ratio
SIG	Speech signal quality (DNSMOS P.835 sub-score)
SIR	Signal-to-interference ratio
SI-SDR	Scale-invariant signal-to-distortion ratio
SLT	IEEE Workshop on Spoken Language Technology
SNR	Signal-to-noise ratio
STFT	Short-time Fourier transform
TF Map	Time-frequency map
TSE	Target speaker extraction
VAD	Voice activity detection
WER	Word error rate

Symbols

Signals and transcriptions

x	Mixture signal
s	Target reference signal
$\hat{s}$	Model output, the estimated target
d	Interfering speech signal
$y_{(\cdot)}$	Transcription of the subscripted signal
$y_s^\star$	Ground-truth LibriSpeech transcription of the target

Chapter 1

Introduction

Chapter 2

Literature Review

2.1. Band-split Recurrent Neural Network (BSRNN)

Luo and Yu [1] propose the BSRNN model as a frequency-domain separation model that is designed for high sample rate audio, in which the input audio is transformed from its waveform representation into a time-frequency representation using the Short-Time Fourier Transform (STFT). The model then divides the input spectrum into multiple frequency bands that are modelled independently. The original use case for the BSRNN model is music source separation, but in this work, the model is adapted to allow for the extraction of a target speaker from a mixture of speech signals. The development from music source separation to the method of the target speaker extraction (TSE) approach that this work follows came from the following lineage: *(i)* music source separation was adapted to personalised speech enhancement (PSE) by Yu et al. [2]. The authors proposed target speaker extraction in the PSE task by appending a target speaker enrollment module that was designed to suppress any interference. PSE can be summarised as enrollment-conditioned extraction. *(ii)* Zhang et al. followed this framework and proposed a new module for the enrollment conditioning module, namely the Time-Frequency Map (TF Map) [3].

Chapter 3

Methodology

3.1. Data construction

The section describes the construction of the dataset used for this work. It includes a description of the necessity of constructing the data from scratch, the source of the audio used, and the construction of the trials.

3.1.1. Why the data is constructed

There are three main considerations for the dataset that need to be met: *(i)* training requires a clean target signal as a reference for what the extractor should output, *(ii)* the data must possess exact transcriptions of the target signal to allow for transcription-based evaluation, and *(iii)* the data must possess both best case and worst case scenarios in order to interpret the performance of the extractor relative to these scenarios. These considerations limit the available choices of publically available datasets. For inspiration for the construction of the dataset, the 2026 REAL-TSE challenge [4] entries were considered. The organisers of the challenge noted that the contributions to a successful system were attributed to well crafted data.

3.1.2. Source of audio

The audio data was collected from a variety of sources to produce a diverse dataset. Speech was chosen for target and interferer from the LibriSpeech [5] dataset, which is a corpus of read English speech derived from audiobooks. The background noise was chosen from WHAM! [6] which is a dataset of real-world noise recordings considering environments such as offices, streets, and cafés.

3.1.3. Trial construction

The trials are structured in a way to replicate the conditions of a real-world target speaker extraction scenario. Due to the difficulty of the TSE task, it was decided that the data should reflect a variety of conditions, but primarily focus on the base case task of a target speaker close to a recording device with an interfering speaker in a similar proximity and faint background

noise. The difficulty came in constructing the dataset to be realistic enough to simulate a real-world scenario, but also not so difficult that a simple extractor of limited capacity would be unable to learn the task. A visual representation of the trial construction can be found in Figure 3.1.

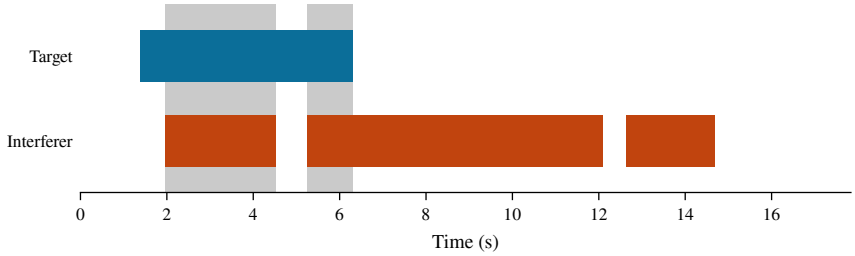

Figure 3.1: Visual representation of the trial construction. The target and interferer are drawn from LibriSpeech, and the noise bed is drawn from WHAM!. The mixture is constructed by adding the three components together, and the enrollment clip is drawn from the same source as the target speech.

A single trial consists of the following components: *(i)* target speech, *(ii)* interferer speech, *(iii)* noise bed, *(iv)* the mixture of the first three components, and *(v)* an enrollment clip of the target speaker. The trial belongs to one of four conditions: both speakers present, only the target speaker present, only the interferer speaker present, and neither speaker present. The mixture is constructed to be 15-20 seconds long. The trials are drawn from two separate regimes: a base case and a difficult case. The base case is constructed to be a simple but realistic scenario, and difficult cases are edge cases that are constructed to be more challenging for the extractor. There are two both male and female speakers in the dataset. To simulate room acoustics, including reverberation, room dimensions, source and microphone positions, the `pyroomacoustics` [7] package is used to simulate the room impulse response. The trials are constructed to be as realistic as possible, but also to be reproducible and controllable. A sample of the room construction can be found in the Appendix B.

Enrollment. [TODO validatae the enrollment clip length and whether it is a single clip or multiple clips] The enrollment clip is a short audio segment of the target speaker. The clip is used to provide the extractor with a reference of what the target speaker sounds like. The clips are taken from the same source as the target speech, which is LibriSpeech. To avoid bias in the extractor, the enrollment clip is first attempted to be drawn from a different book reading. If this is unsuccessful, then the clip is drawn from a different chapter of the same book. Finally, if this is also unsuccessful, then the clip is drawn from a different utterance of the same chapter. The reason for this is to avoid the extractor memorising the target speech by learning to recognise the content rather than the *manner* that the target speaker speaks. The distribution of enrollment clips across the dataset is shown in Figure 3.2. Again to avoid bias, the enrollment clips are simulated to come from different recording devices. This is done by applying equalisation (EQ) augmentation to the enrollment clips by applying a random EQ

filter to the clip with a probability of 0.5. This technique was suggested by the CARTSE [8] submission for the 2026 REAL-TSE challenge [4].

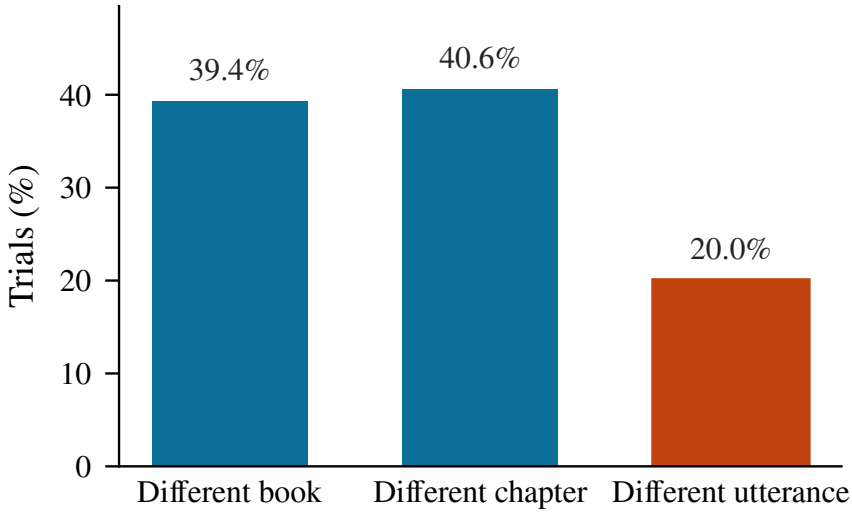

Figure 3.2: Distribution of the source of the enrollment clips across the dataset.

Parameter distribution choice. The Signal-to-Interference Ratio (SIR) is chosen uniformly from the range of -10 to 10 dB, and the Signal-to-Noise Ratio (SNR) is chosen uniformly from the range of 0 to 20 dB. These two ratios simulate the relative loudness of the target speaker to the interferer and background noise, respectively. A negative number for the SIR indicates that the interferer is louder than the target speaker, and the lower the SNR, the louder the background noise is relative to the target speaker. The SIR was chosen to be symmetric around $0\,\mathrm{dB}$ after experimentation found that if the target was always (or majority of the time) louder than the interferer, then the extractor would just learn to follow the speech that was loudest and hence ignore the conditioning of the enrollment audio. This defeats the purpose of the task hence the stated requirement.

The reverberation of each room is set by a reverberation time, T_{60}, which is defined to be the time it takes for sound to decay by $60\,\mathrm{dB}$. The larger the value the more reverberant the room is. T_{60} is drawn from a uniform distribution between $0.25\,\mathrm{s}$ to $0.6\,\mathrm{s}$, which are values that span a carpeted office to a larger room with hard surfaces. To enforce realism, the absorption rate of the walls is also considered. The absorption rate is considered using the Sabine relation [9]. The relation connects the physical properties of a room to its volume, its surface area and the absorption of its surfaces, so once the dimensions and the T_{60} have been drawn the absorption is fixed by them and is solved for rather than sampled. A draw is rejected and replaced if the requested T_{60} is not achievable in a room of the drawn size, which is also the reason for the lower bound of $0.25\,\mathrm{s}$.

The remaining parameters and their distributions can be found in the Appendix C

Target-absent trials. The extractor needs example trials to learn to rely on the conditioning of the enrollment audio to identify target speech. The target-absent trials support this idea by having the model learn to stay silent when the enrollment speaker is not present. The split is as follows for these trials: 49.5% both speakers present, 25.5% target only, 19.6% interferer only and 5.4% noise only.

Splits. The dataset is split into a training and validation sets. Each of these sets have their own distinct set of speakers such that there is no overlap of speakers between the sets. The training set consists of 9955 trials, consisting of 1172 unique speakers. The validation set consists of 200 trials drawn from 40 speakers. The idea behind this is to ensure that the model does not memorise speakers, but rather to learn to extract the target speaker based on the enrollment clip. Finally, in a final evaluation holdout set, there are no target-absent trials. The training split was progressively built up in stages from 1989 to 4976 and then finally to 9955 trials. This was done to measure the effect of the quantity of data on the performance of the extractor. The final evaluation holdout set consists of 103 trials.

Limitations. The dataset is constructed to be as realistic as possible, but it is still a simulated dataset. The dataset is restricted to two speakers, which can confound the extractor to expect the target to be present if two speakers are present. Secondly, the combined audio forms *read* speech and not *conversational* speech. This means that the resulting audio does not have natural turn-taking, or conversational dynamics. These dynamics are simulated through the overlap of the target and interferer, but it is not a perfect simulation. Next, the dataset is restricted to English speech, and shoebox rooms only.

3.2. Baseline model architecture

3.2.1. Short-time Fourier transform

When speaking, the audio contains energy. The energy is not evenly distributed across frequencies, and in normal speech it is not evenly distributed across time either. The energy is concentrated in particular frequency bands by the unique resonances of the speaker's vocal tract, and is also concentrated in time by syllables, words and phrases. Audio models therefore want to leverage that structure, and require the input to be presented in a representation that allows for those concentrations to be observed. The waveform itself cannot represent that structure as it is a one-dimensional signal and is not aligned with the human perception of sound.

The short-time Fourier transform (STFT) produces that representation. The waveform is cut into short overlapping segments. These segments are used to compute the Fourier transform, which is used to decompose the signal into its frequency components. These overlapping

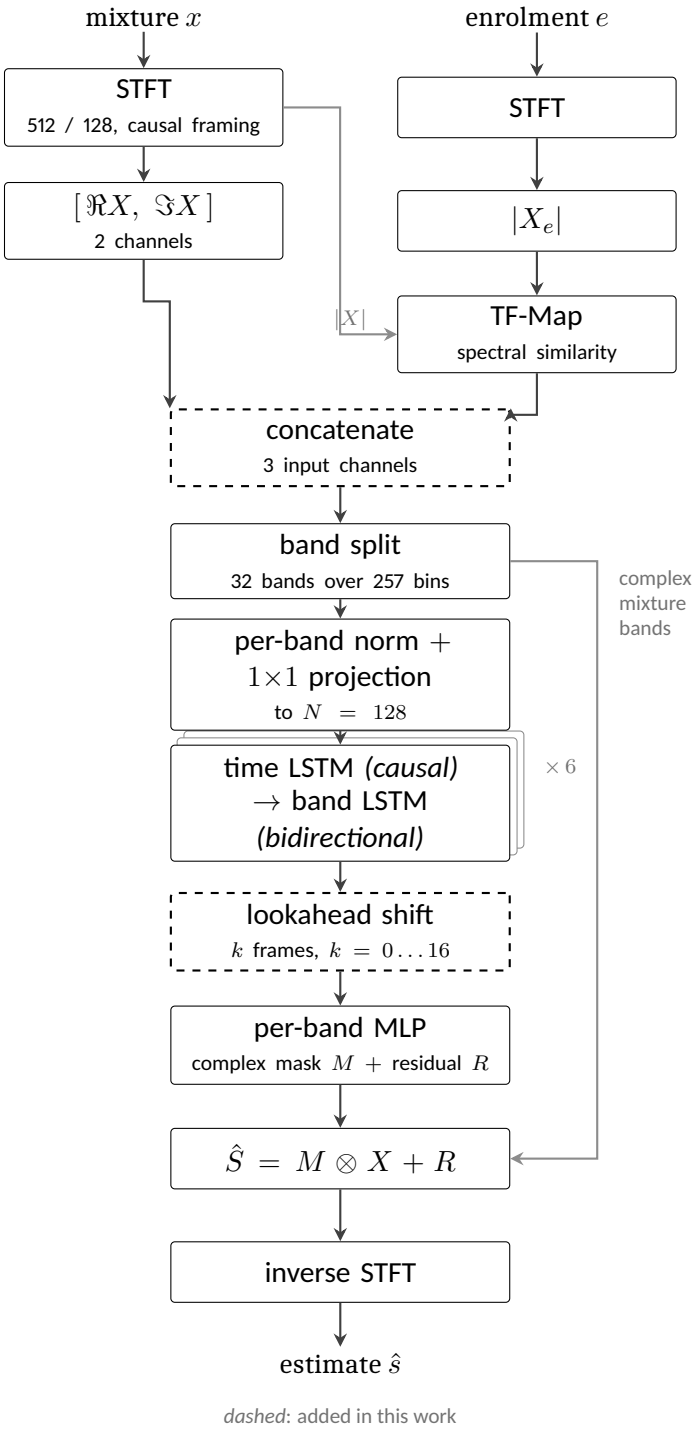

Figure 3.3: The extractor as implemented. Band splitting and the interleaved band-level and sequence-level modelling follow [1]; the mask-plus-residual estimator follows [2]; TF-Map conditioning follows [3]. The causal time path, the lookahead shift and the third input channel are this work's.

segments are called frames, and the idea is to pass each of these segments through the Fourier transform by first multiplying the segment by a window function. The window function, which is typically a Hann window, tapers at the ends such that when multiplied, silences the ends of the segment to isolate the middle. After this a discrete Fourier transform is taken of each segment. The result is one complex number per frequency bin per segment: for a real input x, the coefficient at frequency bin f and frame t is

$$X(f,t) = \sum_{n=0}^{N-1} x[tH+n]\, w[n]\, e^{-j2\pi fn/N}, \tag{3.1}$$

where j is the imaginary unit ($j = \sqrt{-1}$), N is the window length in samples, H is the hop by which successive windows advance, and $w[n]$ is the window function. The hop is what makes the segments overlap: consecutive frames share $N - H$ samples, so a feature that falls near the edge of one frame sits comfortably inside the next. For the implementation in this work, a window size of $N = 512$ is selected, with a hop of $H = 128$ samples at $16\,\mathrm{kHz}$ gives a $32\,\mathrm{ms}$ window shifting by $8\,\mathrm{ms}$. These are the values Yu et al. [2] specify for their $16\,\mathrm{kHz}$ model. Secondly given the constraint of the streaming budget of $200\,\mathrm{ms}$ to $300\,\mathrm{ms}$, the framing consumes well under a quarter of that budget, and the remaining headroom allows for budget to be spent on the lookahead of Section 3.2.5. Finally, the approach in this work forms the frames without centring creating the causal framing. The reason for this is that centring each frame on its own timestamp makes frame t consume $N/2 = 256$ samples – $16\,\mathrm{ms}$ – of audio recorded *after* t. Since the objective is to create an online, streaming system and thus the model should not depend on future input. To account for the beginning samples of the audio, the signal is padded by $N - H$ samples at the start, which is a kind of cold start of a live system whose input buffer begins empty.

Each coefficient carries two quantities. Its magnitude $|X(f,t)|$ is how much energy the signal has in that frequency band at that time, and its phase $\angle X(f,t)$ is where in repetition of the waveform the signal is at that time. The magnitude is what the model uses to determine which parts of the mixture belong to the target speaker, and the phase is what the model uses to reconstruct the waveform from the modified magnitude. The complex spectrogram is kept separately, since the mask of Section 3.2.6 is applied to it rather than to the network's internal features.

3.2.2. Band plan

The spectrogram of Section 3.2.1 has 257 outputted frequency bins, each covering an equal slice of $31.25\,\mathrm{Hz}$. Treating all 257 alike would waste the model's capacity, because speech does not spread itself evenly across them. Most of the energy, and nearly all of the structure that distinguishes one voice from another, sits at the lower frequencies: the pitch of a voice and the resonances that give a vowel its identity all fall below about $4\,\mathrm{kHz}$, and energy falls away

steadily above that. The bins near the top carry far less information and thus would be wasteful to model with the same level of detail as the bins near the bottom.

A band plan is the decision of how to group those bins. Bins are gathered into contiguous groups, called bands, and each band is then handled separately by the network: it gets its own normalisation and its own projection, so a band is compared against itself over time rather than against the loud low-frequency bands next to it. Grouping few bins per band keeps fine detail. Secondly grouping many bins into one band summarises coarsely but costs far less. A band plan therefore decides where the model's resolution is spent.

The plan in this work is 32 bands over the 257 bins: fifteen bands of $100\,\mathrm{Hz}$ covering $0\,\mathrm{kHz}$ to $1.5\,\mathrm{kHz}$, ten of $200\,\mathrm{Hz}$ covering $1.5\,\mathrm{kHz}$ to $3.5\,\mathrm{kHz}$, five of $500\,\mathrm{Hz}$ covering $3.5\,\mathrm{kHz}$ to $6\,\mathrm{kHz}$, and the remaining $6\,\mathrm{kHz}$ to $8\,\mathrm{kHz}$ as one band. Figure 3.4 shows it against a real mixture in the training set. The effect is a factor of twenty between the finest and coarsest resolution. This plan follows the implementation from Zhang et al. [3] for their $16\,\mathrm{kHz}$ target speaker extraction model and used by the challenge baseline built on it, but no reason is given there for these particular boundaries, and no explanation is given in previous papers. The original band splitting was designed for music [1], where the sources are instruments rather than voices. It is adopted here as the baseline so that this work's results are comparable with that lineage, and it is recorded as an unjustified inheritance rather than a motivated choice.

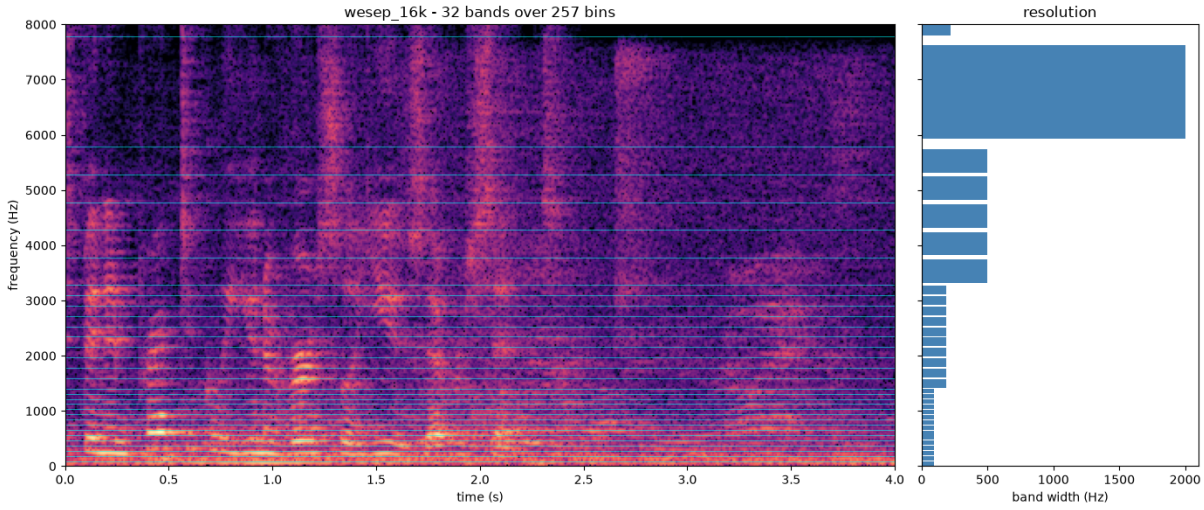

Figure 3.4: The band plan used here, shown against a four-second mixture from the training set. Left: the band boundaries drawn over the spectrogram, closely spaced at the bottom where the speech structure is and widening upwards. Right: the width of each band in hertz. The lowest fifteen bands are $100\,\mathrm{Hz}$ wide; the single highest spans $2\,\mathrm{kHz}$. This band plan follows the one used by Zhang et al. [3] for their $16\,\mathrm{kHz}$ model.

[GRANT: TODO - Possible experiment] *Because the plan is a free parameter that nobody has justified, it is also implemented as a configurable table rather than fixed in code, with alternative plans available for comparison.*

Why the spectrum stops at $8\,\mathrm{kHz}$. The audio in this work is sampled at $16\,\mathrm{kHz}$, meaning 16,000 amplitude measurements per second. When measuring a wave you need at least two measurements per cycle, one for the peak and one for the trough of the wave. Therefore the fastest possible wave that $16\,\mathrm{kHz}$ sampling can represent will complete 8,000 cycles per second. Anything faster cannot be distinguished from a slower wave passing through the same measured points. Therefore, the largest possible range that can be represented by the 257 bins have to span $0\,\mathrm{kHz}$ to $8\,\mathrm{kHz}$ rather than $0\,\mathrm{kHz}$ to $16\,\mathrm{kHz}$.

3.2.3. Per-band normalisation

The bands defined by Section 3.2.2 are not comparable with one another. Speech energy falls away by roughly $6\,\mathrm{dB}$ per octave, so the lowest bands carry orders of magnitude more energy than the highest. Therefore, if all 257 bins were scaled together, there would be a severe imbalance where the high bands would be underrepresented. This would then hurt the performance of the model as the high bands would be diluted and would contribute almost nothing to the gradient, and the model would in effect only learn to separate the bottom of the spectrum. Giving each band its own scaling is what prevents that. A band at $6\,\mathrm{kHz}$ is then judged against its own typical level rather than against the bands beneath it.

This module does two things to each band independently, following [2]. First it normalises the band, using a layer normalisation applied across the channels of that band. Secondly, because the bands must be the same size before they can be stacked together, a projection is required to map the band to a fixed width. Since there are varying sized and independent bands, each band has its own projection, which is a learnable linear layer that maps the band to a fixed width of $N = 128$ values. Every band is given the same structure but its own weights, so nothing is shared between bands and each is free to learn a representation suited to its own frequency range. The result is a stack of 32 bands, each with 128 features, which is then passed to the next module.

Normalisation is applied across the values of a band *within a single frame*. Each frame is therefore normalised using only itself. This is a deliberate design choice that opposes Yu et al. [2], who use batch normalisation for their online configuration. The opposition is for the following reasons: *(i)* although batch normalisation will work for online models as indicated by Yu et al. and is causal, there are two different operations of batch normalisation: training and inference. During training, the mean and variance are computed from the current batch, but during inference, these statistics are computed from the entire training set. The function that is optimised during training is therefore not the function that is deployed. *(ii)* That gap is unusually wide here. A batch is twelve four-second crops drawn at random from a corpus spanning a range of reverberation times, signal-to-interference ratios and noise levels, and in which roughly three crops in ten contain no target speech at all. The statistics of a batch therefore depend on what was drawn, and a silent crop normalised alongside loud ones is not

normalised as it would be at inference. Because normalisation is per band, the effect is worst in the high bands, where the energy is smallest and the variation between batches is largest relative to the signal. *(iii)* Single frame normalisation carries no state and is unaffected by the amount of audio processed, so a four-second training chunk is normalised exactly as a stream that has been running for a minute.

3.2.4. Residual LSTM block

The idea of the residual LSTM block is to model the sequential structure of speech such that the model can leverage patterns that unfold over time. There are two kinds of sequential structures that are modelled here. The first is the sequence over time, which is the natural order of speech. The second is the sequence over frequency. Think of the time being unidirectional, as the speech only makes sense to unfold forward in time, but the frequency is bidirectional, as all of the bands of a frame are available at the same instant and can be traversed in either direction. The block is used for both kinds of modelling, with the same structure but different parameters.

The recurrent layer is what carries information forward where this work leverages a long short-term memory layer, or LSTM. This is a recurrent layer with an internal mechanism for deciding what to keep in that summary and what to discard, which is what allows it to retain information over hundreds of frames rather than only the last few. This structure also suits the deployment target directly. At inference time, since the model is running in a streaming fashion, the LSTM's internal state is preserved between frames and information can be carried forward indefinitely. The LSTM is therefore a natural fit for the task, and it is used in the same way for both the time and frequency sequences.

The block is made of three steps applied in order, following [1]. The input is first normalised, then passed through the LSTM, and the LSTM's output is then mapped back to the input's width by a learned projection. The final addition is a residual connection and is inspired by the original ResNet [10] architecture. This allows the gradient an unobstructed path back through the computation stack of all the blocks, which matters because six of these blocks are stacked in Section 3.2.5. This is important as the LSTM is a deep recurrent layer, and the gradient can vanish or explode as it passes through the LSTM's internal state. The residual connection allows the gradient to bypass the LSTM entirely, which is what allows the model to train when the gradient vanishes within the LSTM.

[GRANT: TODO] - Still might need to increase the size of the model so this is not final!!!

What is used here. The LSTM has a single layer with a width of 192, taken from Yu et al. [2], whose $16\,\mathrm{kHz}$ model specifies that width. This is deliberately narrower than the reference implementation of the same architecture, which uses twice the feature width, or 256. Because the LSTM's width differs from the 128 features entering the block, the projection is not optional bookkeeping: it is what makes the addition dimensionally possible at all.

The block is written to be indifferent to what its sequence axis represents. It receives a batch of sequences and processes them; whether those sequences run along time or along frequency is decided by the caller. This is what allows the same block to be used for both kinds of modelling in Section 3.2.5, with one critical difference between the two uses. Along time the LSTM runs forward only, because a backward pass over time reads frames that have not yet arrived. Along frequency it runs in both directions, which costs nothing in latency, since all the bands of a frame are available at the same instant.

3.2.5. Band and sequence model

The band and sequence model is the implementation of the residual LSTM block of Section 3.2.4 in a stack of six blocks, using the input from Section 3.2.3. After the per-band normalisation and projection, there are 32 bands of 128 features each. The model is now tasked for the two kinds of sequential modelling with the residual LSTM blocks as previously mentioned: time and frequency.

Each of these blocks is designed to make two passes. The design is to first make a pass over the time dimension (unidirectional), and then make a pass over the frequency dimension (bidirectional). This is considered as follows:

- **Time**: The recurrence goes forward only, as a backward pass would assume knowledge of future frames that have not yet arrived.
- **Frequency**: The recurrence goes over the bands of a frame in both directions. This pass is what lets the bands inform one another: a voice lays down a harmonic pattern across many bands at the same instant, so whether a given band belongs to the target is often not decidable from that band alone. Frequency itself is not a causal dimension. This can be reasoned by the fact that all the bands of a frame are all available simultaneously so the bidirectional recurrence does not introduce any latency.

Pairing the two passes in a single block is the design of Luo and Yu [1], who motivate it by analogy with dual-path recurrent networks for speech separation rather than by an argument specific to bands. Stacking six such blocks follows Yu et al. [2], where their $16\,\mathrm{kHz}$ model uses that specific number of blocks. The effect of stacking the blocks is to allow for the accumulation of evidence over longer time spans. The evidence can then be shared across bands of the next block and so forth.

Lookahead. The model is set to allow for a lookahead of k frames, ranging from 0 to 16, which costs k hops of additional latency, $8k$ milliseconds. The REAL-TSE challenge, an IEEE SLT 2026 satellite challenge [4] counts lookahead frames as part of the latency budget and does not consider it as a parameter that will break the online requirement. It is rather an expense for the overall latency budget. CARTSE [8] uses no lookahead frames at all, so its

algorithmic latency reduces to the window minus the hop. The challenge overview reports no comparison of lookahead against the overall quality of the TSE task, so whether it helps is an open question rather and **is treated as an experimental question**. ¡- [TODO - Grant: Need to test this lookahead or remove if not applicable]

The main idea behind using lookahead is that it can be reasoned that often evidence for whether a frame belongs to the target speaker might arrive late. If the model waits slightly to see what comes next for a brief moment, it does not need to commit to a decision immediately. At $k = 0$ the model has to commit to a decision.

[GRANT TODO] - NEED TO TEST THIS LOOKAHEAD *The baseline is trained at $k = 0$ so that the fully causal model is what any larger setting is measured against. The additional delay is treated as an experimental question in Chapter 4 rather than assumed here.*

It is implemented by pairing each frame with a later summary of the sequence. The model still processes every frame as it arrives. What changes is which of the stack's output frames the estimation module will read. To produce the mask for frame t it reads the state the network produced at frame $t + k$ rather than its state for t. Since the recurrence for time only goes forwards, the model will consider everything that was heard up to frame $t+k$ when considering the estimation for frame t.

3.2.6. Estimation module

The estimation module is responsible for producing an output spectrogram that contains the estimated target speaker's speech. This is done by creating an output network per band to reconstruct the target speaker's speech from the features produced by the band and sequence model. However, there are two methods to do this: either to predict the output spectrogram directly, or to predict a mask that is applied to the mixture. The latter is the approach taken here, following Yu et al. [2].

Yu et al. realised that the mixture audio itself already contains the target's speech, and thus it would be more efficient to mask the mixture rather than predict the output spectrogram directly. This mask is a number for each time-frequency bin, which the mixture is multiplied by.

A mask alone is not enough however as multiplication can only ever scale what a bin already holds. In the cases where the interferer completely dominates a bin no multiplier recovers the target as the multiplication operation would not be able to scale the drowned out target. To solve this, the estimation module makes use of a residual spectrogram and is predicted directly and appended. Yu et al. [2] introduce it for exactly this reason, to absorb what the mask alone produces as artefacts. It is useful to not bound this mask and is what the original implementation encouraged. The estimate for a band is then

$$\hat{S} = M \otimes X + R, \tag{3.2}$$

where $\otimes$ is the complex product, taken bin by bin. The X that the mask multiplies is the original

complex mixture and is taken as is from the band split and not from the output of the model's features. The features are used only to *decide* how to mask the input. The conditioning feature of Section 3.2.7 therefore never enters the signal path either.

Each band has its own two-layer output network: a normalisation, then a projection to a width of 384 with a $\tanh$ activation. Finally we have two output layers reading from it, one producing the mask and one the residual. The width of 384 is the value Yu et al. [2] specify.

The mask output layer has to output four pairs of real and imaginary parts for each frequency bin. Two of them are taken as the mask values, and the other two are passed through a sigmoid, giving a number between zero and one that each mask value is then multiplied by. This arrangement is a gated linear unit [11], and it is what both source papers specify for this layer [1, 2].

3.2.7. Time-frequency map

The TF Map is the enrollment-conditioning module to turn normal speech enhancement into the TSE task. The idea behind this module is for the model to take an enrollment of the target speaker, and use the unique audio signature of that speaker to condition the model to extract only that speaker from the mixture. The general process includes processing the enrollment clip through the STFT module, extracting the magnitude of the enrollment, and then breaking up the spectrogram into basis vectors. These basis vectors are then compared to the mixture's spectrogram using cosine similarity.

Concretely, let $|X| \in \mathbb{R}^{F\times T}$ be the magnitude spectrogram of the mixture and $|X_e| \in \mathbb{R}^{F\times T_e}$ that of the enrollment, and write $|X_t|$ and $|X_{e,j}|$ for one frame of each. The module proceeds in three steps. *First*, every frame of both spectrograms is scaled to unit length,

$$\tilde{x}_t = \frac{|X_t|}{\big\||X_t|\big\|}, \qquad \tilde{e}_j = \frac{|X_{e,j}|}{\big\||X_{e,j}|\big\|}. \tag{3.3}$$

Second, each mixture frame is compared against every enrollment frame by cosine similarity. Because (3.3) has already removed the length of each frame, the cosine of the angle between two frames is simply their inner product:

$$S_{t,j} \;=\; \cos\theta_{t,j} \;=\; \frac{\big\langle |X_t|,\; |X_{e,j}| \big\rangle}{\big\||X_t|\big\|\,\big\||X_{e,j}|\big\|} \;=\; \big\langle \tilde{x}_t,\; \tilde{e}_j \big\rangle. \tag{3.4}$$

This is what makes the comparison of spectral *shape* rather than of loudness: a quiet frame and a loud frame with the same spectral pattern are identical under (3.4). Note also that magnitudes are non-negative, so $S_{t,j} \in [0, 1]$ for every pair.

Third, the similarities of a mixture frame are turned into weights over the enrollment, and

those weights are used to blend the enrollment frames,

$$H_{t,j} = \underset{j}{\mathrm{softmax}}\left(S_{t,j}\right), \qquad M_t = \sum_{j=1}^{T_e} H_{t,j}\,\tilde{e}_j, \tag{3.5}$$

following [3]. The softmax runs along the enrollment index j, so each row of H sums to one and describes how to blend the enrollment frames for one instant of the mixture. M_t is a guessed spectrum: for each mixture frame, a weighted average of all the enrollment frames, with the weights given by H. It is a rough estimate of the shape of the target speaker's spectrum at that instant.

M_t describes only that shape. It is built entirely from enrollment frames but on its own it says nothing about whether the target is speaking. Note also that the mixture's magnitude has not been used at any point so far: (3.3) removed it before the comparison in (3.4) was made. The final step is to address the energy component. The estimate is scaled to unit length and the mixture's spectrum is projected onto it which measures how much of the mixture's actual energy lies in the pattern the enrollment predicted:

$$f_t = \left\langle\, |X_t|,\, u_t \,\right\rangle u_t, \qquad u_t = \frac{M_t}{\|M_t\|}, \tag{3.6}$$

The feature therefore points in the direction the enrollment predicts and has the magnitude of however much of the mixture actually lies in that direction. Where the mixture at that instant looks like the target, it is large otherwise small. It is this that the model learns to use to identify which parts of the mixture belong to the target speaker.

Where it enters and what it costs. The result is a quantity of the same shape as the spectrogram itself, one value per frequency bin per frame. It is therefore attached as a third input channel alongside the real and imaginary parts, before the band split, so that it is divided into bands and projected by the same machinery as the rest of the input. Nothing else in the architecture changes. Because the extra channel only widens the input of the per-band projections, the entire conditioning path costs roughly $33\,000$ parameters, under half a percent of the model, and it adds no learned parameters of its own at all: (3.3) through (3.6) contain nothing to train.

3.2.8. Objective function

[TODO UPDATE WITH NEW LOSS FUNCTION]

The objective has to express two different requirements, because the training data contains two different situations. Where the target speaks, the output should match the target signal. Where the target is silent as the result of either the trial contains no target speech at all, or because the four-second crop happened to land in a gap between the target's utterances, there is no signal to match, and the correct output is silence. A single loss cannot serve both use cases

and so the objective is therefore split, and each crop is scored by the branch that applies to it.

Throughout, x is the mixture, s the target reference and $\hat{s}$ the model's output, all in the time domain over one training crop.

Target present. The first term measures how much of the output is the target and how much is everything else, following [8]. Importantly, the loss needs to be invariable to the overall volume as if the $\hat{s}$ is of the correct shape, it should not be penalised for being loud or quiet. Therefore the first step is to find the single gain that best explains the output as a scaled copy of the target, which will be calculated as a projection,

$$\alpha = \frac{\langle \hat{s},\ s \rangle}{\|s\|^2}, \qquad s_{\text{proj}} = \alpha s, \tag{3.7}$$

and then comparing the energy of that scaled target against the energy of whatever the output contains beyond it. The loss is modelled after the Scale-Invariant Signal-to-Distortion Ratio (SI-SDR). The loss is therefore,

$$L_{\text{pres}} = -10 \log_{10} \frac{\|s_{\text{proj}}\|^2}{\|\hat{s} - s_{\text{proj}}\|^2 + \tau \|s_{\text{proj}}\|^2}. \tag{3.8}$$

Lower is better. The loss is secondly in terms of decibels (dB), which is where the logarithm comes from. The τ term in the denominator is a floor. Without it a perfect output would score $-\infty$, and the gradient would keep rewarding refinement of crops that are already essentially solved. With $\tau = 10^{-3}$ the best attainable score is $-30\,\mathrm{dB}$. This constant follows the suggestion from CARTSE [8]. The reasoning is that an output whose residual error already sits $30\,\mathrm{dB}$ below the target is close enough to be treated as solved, so it would be wasteful to keep spending gradient on it. Two properties of this measure should be stated plainly, because they are what the second term exists to compensate for. The first is that everything the output contains beyond the target is collected into one denominator. Residual interfering speech, residual background noise and artefacts the model itself introduced all count identically per unit of energy, although they are not equally damaging: a fragment of the interferer left in the output can be transcribed downstream as words the target never said, whereas a diffuse artefact of the same energy usually cannot. The measure has no way to express that difference. The second is that it is an energy ratio summed over the whole crop, so it is dominated by whatever is loudest in it. Speech energy sits overwhelmingly in the low frequencies and in vowels, which means a model can score well on (3.8) while reconstructing fricatives and the upper spectrum poorly. Those are the parts that carry consonant identity, and therefore much of what makes speech intelligible rather than merely present.

Multi-resolution spectral term. Equation (3.8) is a single number computed over the whole waveform, which makes it ignore the location of errors in the spectrum. Since it cannot express

where in the spectrum the error lies, a model that reconstructs the loud low bands well and the quiet high bands poorly is barely penalised and secondly since being scale-invariant, it does not constrain the output's absolute loudness at all. The idea is to use a loss that considers multiple resolutions of s and $\hat{s}$. By considering different granularity of the spectrogram, the model is encouraged to reconstruct both the low and high frequency bands well. A spectral term allows for this following [2]:

$$L_{\text{MR}} = \frac{1}{I}\sum_{i=1}^{I}\left(\left\| |S_i|^p - |\hat{S}_i|^p \right\|_1 + \left\|\Re S_i - \Re\hat{S}_i\right\|_1 + \left\|\Im S_i - \Im\hat{S}_i\right\|_1\right), \tag{3.9}$$

where S_i and $\hat{S}_i$ are the spectrograms of the target and the output under the ith of I analysis window lengths, and each norm is averaged over its time-frequency coefficients.

Three choices inside (3.9) are worth stating. The exponent $p = 0.3$ compresses magnitudes, which will highlight the quieter components relative to loud ones so that an error in a low-energy region is not diluted just because it is quite. Without the compression the term would be dominated by the same loud low bands that already dominate (3.8). The real and imaginary terms are left uncompressed and they are what make the term sensitive to phase: a magnitude-only loss would be indifferent to the phase that the complex mask of Section 3.2.6 is free to change. Finally the term is evaluated at several window lengths rather than one, because a single analysis window can hide errors at its own scale.

The window lengths used are 8, 16, 32 and $64\,\text{ms}$. This is with respect to the model's own $32\,\text{ms}$ framing. The model builds its output by masking $32\,\text{ms}$ frames and overlapping them, so the places that most errors would occur can be expected to exist at frame boundaries. The $8\,\text{ms}$ and $16\,\text{ms}$ windows resolve this and the $64\,\text{ms}$ window catches harmonic structure that the short ones blur.

Target absent. When there is no target speaker present in the mixture audio, the model should output silence and suppress all other noise. The model should therefore penalise any output that is not silent. This is modelled after a modified version of the absolute level loss from CARTSE [8], where in this works the loss now contains the denominator term.

$$L_{\text{abs}} = 10\log_{10}\frac{\|\hat{s}\|^2 + \tau\|x\|^2}{\|x\|^2}. \tag{3.10}$$

Including the denominator term makes the loss relative to the input, which is directly readable: $0\,\text{dB}$ means the output is as loud as the mixture, that is, the model did nothing; $-10\,\text{dB}$ means the interfering speech was suppressed by $10\,\text{dB}$; and the same τ floors the term at $-30\,\text{dB}$, beyond which the output counts as silent. A positive value means the model amplified the mixture, which is a direct failure. The inclusion of the denominator also allows for scale invariance, which puts the loss in the same units as the loss for the target present case.

Combining the two. Each term is averaged over the crops it applies to, and the two averages are weighted against one another:

$$L = (1 - w) \underset{\text{target present}}{\text{mean}} \Big[L_{\text{pres}} + w_{\text{m}} L_{\text{MR}} \Big] + w \underset{\text{target absent}}{\text{mean}} \Big[L_{\text{abs}} \Big]. \tag{3.11}$$

Averaging within each branch rather than over the batch as a whole matters more than it appears to. Under a single batch-wide mean, the share of the gradient belonging to the silent crops would be whatever fraction of the batch happened to be silent, so the balance between the two requirements would fluctuate from step to step on the luck of the draw. Under (3.11) that balance is fixed by w.

Which branch a crop belongs to is decided from the audio of the cropped target itself, not from the trial's label. A trial in which the target does speak can still yield a crop containing no target speech, and this occurs in $5.8\,\%$ of crops. Routing such a crop to the present branch would put an all-zero reference into (3.7), where α becomes $0/0$. the implementation asserts against this rather than relying on the labels agreeing.

Weights. [TODO This doesnt make sense]

The weight w on the silent branch is derived rather than chosen. Silent crops make up a measured 0.297 of the training crops, and the objective is intended to weight silence at twice its frequency in the data, following [8]. Carrying that through gives $w = 0.458$, which reproduces the same focus on silence as the source objective, but using the actual frequency of silence in this project's training data rather than the source's.

The weight w_{m} on the spectral term is derived from the target present and absent losses. The reason for this term is equations (3.8) and (3.9) are in different units. L_{pres} is in units with respect to a ratio in decibels, and L_{MR} is the mean absolute difference between compressed magnitudes. Therefore the weighting term w_{m} will reconcile the two terms to be on the same scale. This is done by evaluating on the output that does nothing at all, $\hat{s} = x$, over 300 training crops which will simulate the scenario for the average losses for an untrained model. The median values for L_{pres} and L_{MR} are -5.91 and 0.184 respectively. The aim is for the spectral term (Multi-Resolution) to be roughly $30\,\%$ of the present branch at the start of training. The calculation then becomes

$$w_{\text{m}} = \frac{0.3 \times |L_{\text{pres}}|}{L_{\text{MR}}} = \frac{0.3 \times 5.9088}{0.1842} = 9.62$$

Differences with the source objective. The present and absent terms are inpsired from the CARTSE system [8], the online-track submission whose task most closely matches this one, but they are slightly modified. Table 3.1 lists the differences. The first is the most consequential: as published, the floor in (3.8) is taken on $\|s\|^2$ rather than on $\|s_{\text{proj}}\|^2$, which breaks the scale invariance the term is built on. With that form, an output of the correct shape scaled by a gain g scores better the louder it is made and it is not bounded. Flooring on the projection instead

makes numerator and floor scale together, so the two cancel and the score is flat with respect to gain.

Table 3.1: Departures from the objective of [8] as published. Each is recorded with its reason in the project's decision log.

	As published	As used here
1. Floor in (3.8)	On $\tau\|\|s\|\|^2$, which is not scale-invariant	On $\tau\|\|s_{\text{proj}}\|\|^2$, so that gain cancels
2. Scale of (3.10)	Unnormalised, so the value shifts with input loudness	Divided by $\|\|x\|\|^2$, so $0\,\text{dB}$ means "did nothing"
3. Averaging over the batch	One mean over all crops in the batch	A mean within each branch, so the balance is set by w alone
4. Number of weights	A separate weight on the silent term	Folded into w; under branch-wise means the two are one degree of freedom
5. Silence weight	Twice the absent rate of the source corpus	$w = 0.458$, the same emphasis at this project's measured 0.297
6. Windows in (3.9)	All at or above the model's frame length	8, 16, 32 and $64\,\text{ms}$, slightly lower and higher than the model's $32\,\text{ms}$ frame length

3.3. Metrics

This section describes the evaluation metrics used to assess the performance of a trained target speaker extraction model. These metrics are where the main contribution of the work lies. The particular use case of the TSE model designed in this work is to extract target speech, from a mixture of target and interfering speech, where the model has to be setup to handle the following cases:

- **Case 1:** Target speech is present, interfering speech is present, and background noise is present.
  - **Case 1a:** Interfering speaker interrupts the target speaker, i.e. the target speaker is cut off by the interfering speaker.
  - **Case 1b:** Interfering speaker speaks over the target speaker, i.e. the target speaker is still speaking while the interfering speaker is also speaking.
  - **Case 1c:** Turn-taking, i.e. the target speaker and interfering speaker take turns speaking, with no overlap and space between them.
  - **Case 1d:** Target speaker louder/softer than interfering speaker, and speaks for longer/shorter than interfering speaker.

- **Case 2:** Target speech is present, interfering speech is absent, and background noise is present.
- **Case 3:** Target speech is absent, interfering speech is present, and background noise is present.
- **Case 4:** Target speech is absent, interfering speech is absent, and background noise is present.

Since the main task is intelligibility of the target speaker and not necessarily the *quality* of the target speaker, the metrics that are designed in this section aim to reflect that objective. The metrics are designed for the purposes of using the BSRNN model in conjunction with a live speech-to-speech model, such as Gemini Live [12]. Such a model is not a transcriber: it consumes audio through a learned encoder trained over a far wider distribution than read speech, and it is prompted rather than decoded. The metrics are therefore meant to show how well the model will translate as part of a live conversational system, and hence are designed to be used in conjunction with that system.

3.3.1. Trial definitions

Each trial passed through the model at evaluation produces a 4-tuple $(y_x, y_{\hat{s}}, y_s, y_d)$ where y_x is the input mixture speech transcription, $y_{\hat{s}}$ is the estimated target speech transcription, y_s is the target speech transcription, and y_d is the interferer speech transcription. The subscripts follow the audio notation of Section 3.2.8: x is the mixture, s the target reference and $\hat{s}$ the model output, with d denoting the interfering speech and $y_{(\cdot)}$ the transcription of a signal. These transcriptions are created in two ways in this project: (1) by using an automatic speech recognition (ASR) system [13, 14] to transcribe the audio to act as a baseline transcriber, and (2) by making use of a live speech-to-speech model that accepts the audio and outputs a transcription which is prompted with a standard prompt to transcribe the audio found in Appendix A.1. The tuple therefore does not represent necessarily the true transcriptions, but rather what the model/transcriber produces. To avoid confusion, the true transcriptions as found in LibriSpeech will be denoted with a star, e.g. $y_s^\star$

3.3.2. Text normalisation

To standardise the transcriptions created by the different methods, a text normalisation process is applied to the audio transcriptions. Specifically, using the `whisper-normalizer` library [15], the `EnglishTextNormalizer` package, seen in Appendix C of [14], is used to output the transcriptions into lower case, remove punctuation, expand contractions, standardise the spelling of numbers, and remove any non-alphanumeric characters. This is done to ensure that the transcriptions are comparable across different methods of transcription.

3.3.3. Live-model Content Fidelity Word Error Rate

The Live-model Content Fidelity Word Error Rate (LCF-WER) is a metric based off of the arithmetic of the standard Word Error Rate (WER) metric, but applied to a live-model transcription of the audio instead of a standard transcription. It is explicitly labelled as such as the standard WER metric is referenced in the training procedure, and is not associated with the live-model transcription. The standard WER metric is defined as follows:

$$\text{WER} = \frac{S + D + I}{N} \tag{3.12}$$

where S is the number of substitutions, D is the number of deletions, I is the number of insertions, and N is the total number of words in the reference. It is essentially a minimum edit-distance formula, where the number of edits required to change a reference text into the proposed text is divided by the number of words in the reference text. However, there is one flaw in this approach that is required for our metrics to highlight: the broad classes of substitutions, deletions, and insertions do not demonstrate the nuance of how these errors can occur. The LCF-WER is used to represent that fact that the classes of errors are specific to live model transcription.

3.3.4. Interferer Content Rate

The Interferer Content Rate (ICR) is the first metric used to highlight one particular nuance of the LCF-WER metric: a substitution can occur when the model hallucinates a word that is not present in the reference, or the substitution can occur with the model hearing the right word, but from the wrong speaker. The idea for the ICR metric is to highlight the latter case, where the model has heard the correct word, but from the wrong speaker. This will help determine if our model is following speakers, or making up words that are not present in the reference.

Content Words. The ICR metric needs to identify which words are likely to be words that are not filler words that a model can easily hallucinate. Let $\mathcal{C}(\cdot)$ map a normalised transcription to the *set* of content words it contains,

$$\mathcal{C}(y) = \Big\{ w \in \text{words}(y) \ : \ w \notin \mathcal{S} \Big\}, \tag{3.13}$$

where $\mathcal{S}$ is a fixed stopword list, taken from the NLTK English stopword corpus [16]. Function words are removed because almost any two English sentences share them, so an overlap counted over these words would rather count as grammatical issues than content. These content words will be used to make interferer-exclusive bag of words of which will indicate which words the model picks up from the interfering speaker.

Interferer-exclusive content. Firstly, if both the target and interfer say the same word, it is not possible to know which speaker the model heard, thus these words are not considered unique to the interferer. Let $\mathcal{D}$ be the unique content words of the interferer that are not present in the target, and the words that will be considered as leaked from the interferer are the intersection of the model output, $y_{\hat{s}}$ and the interferer-exclusive content words, $\mathcal{D}$,

$$\mathcal{L}(y_{\hat{s}}) = \mathcal{C}(y_{\hat{s}}) \cap \mathcal{D}. \tag{3.14}$$

Write $\ell = |\mathcal{L}(y_{\hat{s}})|$ for the number of words leaked on a trial and $a = |\mathcal{D}|$ for the number of possible unique words to leak. If a trial has $a = 0$, then the trial is not considered in the ICR metric, as there are no unique words to leak.

The contribution that this paper makes is to turn the ICR metric into a rate, as sentences are varying lengths, and thus the amount of leaked words changes with the length of the sentence. The rate secondly helps to answer the following situation: should the model that is finally chosen leak fewer words frequently, or leak more words infrequently. The rating definition is aimed to answer that question, and is defined as follows:

The rate. The rate can be reported at several thresholds. For a set of trials $\mathcal{T}$, and a threshold k of leaked words in a trial, the ICR metric is defined as the fraction of trials in which the number of leaked words ℓ is greater than or equal to k,

$$\text{ICR@}k = \frac{\left|\{\, i \in \mathcal{T}_k \;:\; \ell_i \geq k \,\}\right|}{|\mathcal{T}_k|}, \qquad \mathcal{T}_k = \{\, i \in \mathcal{T} \;:\; a_i \geq k \,\}. \tag{3.15}$$

Lower percentages are better. The rate is used to suggest how many words that are heard by the model that overlap with the interferer such that it is not considered chance if the word was heard.

The eligible set $\mathcal{T}_k$ depends on k. If in a trial there is only one exclusive content word available, it is impossible to achieve ICR@2, so if we keep this trial in the denominator it would skew the results.

3.3.5. Fabrication Rate

The Fabrication Rate (FR) acts as the second part to the ICR metric, where the metric aims to highlight the case where the model has hallucinated a word that is neither present in the target nor the interferer. This metric is again only computable as the LibriSpeech [5] dataset contains the exact transcriptions of the audio, and thus it is possible to determine if a word is hallucinated. The FR metric is defined as follows: Over the N trials with a non-empty response, the statistic is the mean invented count per trial of the number of words in the model output

that are not present in either the target or interferer,

$$\mathrm{FR}_{\text{count}} = \frac{1}{N}\sum_{i=1}^{N}\left|\mathcal{I}_i\right|, \tag{3.16}$$

where $\mathcal{I}_i$ is the set of words in the i-th trial's output that are not present in either the target or interferer and $\nVdash$ is the indicator function for non-membership. The FR metric can also be reported with a threshold k of invented words to consider a trial as a fabrication. The k value is used to determine the number of required words to not be included in either the target or interferer to be considered a hallucination. A word that is mis-transcribed is not necessarily a hallucination, as it is possible that the model heard the word but did not transcribe it correctly. A hallucination is a word that is invented by the model as a by-product of the probabilistic generation process that is a plausible but fabricated word, and not necessarily a mis-transcription of a word that was actually heard. [TODO: I might need a reference for this kind of definition]

$$\mathrm{FR@}k = \frac{1}{N}\sum_{i=1}^{N}\nVdash\left[\left|\mathcal{I}_i\right| \geq k\right], \qquad k = 2 \text{ by convention.} \tag{3.17}$$

3.3.6. Deep Noise Suppression Mean Opinion Score

The Deep Noise Suppression Mean Opinion Score (DNSMOS) is a model based instrument that is used to evaluate the quality of the audio output as if heard from a human listener. The instrument itself is a neural network that predicts what a collection of humans would have rated an audio clip on a scale of 1-5, where 1 is bad and 5 is excellent. In this works, two different DNSMOS protocols are used. Specifically ITU-T P.835 [17] and ITU-T P.808 [18], predicted by the corresponding DNSMOS models [19, 20]. The former returns three metrics namely:

- **SIG:** The quality of the speech in the audio clip.
- **BAK:** The background noise quality of the audio, which measures how much the background noise is affecting the speech in the audio.
- **OVRL:** The overall quality of the audio.

This protocol was used in the SLT 2026 REAL-TSE Challenge [4] to measure how good the different teams TSE models were, but they found that some of the entrants were finding ways to game the metric by making the audio sound better to the model, and not necessarily perform actual target speaker extraction. The challenge organisers therefore made use of the latter protocol in their final evaluation, the P808, which is a single metric from a separately trained model to measure overall quality of the audio clip.

3.3.7. Signal to Distortion/Interference/Artefact Ratios

This section discusses the following ratios: Signal-to-Distortion Ratio (SDR), Signal-to-Interference Ratio (SIR), and Signal-to-Artefacts Ratio (SAR). The ratios are used in the following manner: SDR is used to measure how wrong the output audio is in total, and the SIR and SAR are used to measure in which direction the output audio is wrong. The SIR measures how much of real audio that is not target speech is present in the output audio such as an interfering speaker or background noise, and the SAR measures how much of the output audio is not real audio, such as artefacts from the model. The ratios are defined as follows where y_{tgt} is the target speech, e_{int} is the interfering speech, and e_{art} is the artefacts from the model:

$$SDR = 10 \log_{10} \frac{\|y_{\text{tgt}}\|^2}{\|e_{\text{int}} + e_{\text{art}}\|^2} \tag{3.18}$$

$$SIR = 10 \log_{10} \frac{\|y_{\text{tgt}}\|^2}{\|e_{\text{int}}\|^2} \tag{3.19}$$

$$SAR = 10 \log_{10} \frac{\|y_{\text{tgt}} + e_{\text{int}}\|^2}{\|e_{\text{art}}\|^2} \tag{3.20}$$

which are all measured in decibels (dB) where the higher ratio means better extraction of the target from the other audio in question. When training the model, the Scale-Invariant Signal-to-Distortion Ratio (SI-SDR) introduced by Le Roux et al. [21] is used as the loss function which takes inspiration from the SDR metric and was defined earlier in equation 3.8.

3.3.8. Anchors

3.3.9. Reporting protocol

Chapter 4

Experiments

This section discusses the overall setup to conduct the experiments, including the evaluation protocol, and the experimental setup. The results of the experiments are presented in chapter 5.

4.1. Experimental setup

During the course of experiments, the following setup was used to ensure reproducible and consistent training and evalution of the models.

4.1.1. Training setup

Data pipeline

Model construction: baseline

Model construction: extension

Hyperparameter configuration

4.1.2. Transcription tool setup

Offline ASR model The offline transcriber is the `faster-whisper` implementation [13] of the Whisper [14] model. Of the available options, it was run using the `small.en` package size as this was found to be the best configuration for quick but balanced transcription performance. The model was set to a sampling temperature of zero to ensure deterministic output, and the language was fixed to English.

Live judge The live listener is `gemini-3.7-flash` [12], which was connected to via the `google-generativeai` Python package through the Application Programming Interface (API). The judge, like the ASR model, does not separate anything. The Gemini model is prompted to transcribe the estimated audio from the extractor, and then transcribes what it is able to hear. The judge is held out and never appears in the training loop in any form. This includes training using the judge as a proxy objective, This is to ensure that the judge is not biased towards any particular extractor and represents a fair evaluation of the extractor's performance. The live

model is the tool for which this work is being built for and so it's result is the most important with respect to the overall evaluation of the extractor.

The prompt used can be found in section A.1. It is deliberately worded to avoid instructing the judge to guess or pick a speaker to prevent the judge acting on the extractor's behalf. Secondly, it does not instruct the judge to guess as this would be a form of hallucination, which is a metric that is being measured and thus should not be confounded by the prompt. Finally, since live models by default will return some kind of reponse, the judge is prompted to return a structured response of a status field alongside the transcript. The status field is used to indicate whether the judge heard speech or not, and if it did not hear speech, the transcript is left empty. This allows for a clear distinction for whether the model hallucinates as the status field can be used to determine whether the model heard speech or not.

Speech gating [TODO mention the VAD tool used which was the same in the dataset setup] During experiments, it was common for both transcription tools to invent words when handed audio containing no speech frequently which led to many false positives. To address this issue, a speech gate was implemented to filter out any audio that appeared not to have speech that essentially acts as a pre-filter for the transcription tools. This method was considered for four reasons: *(i)* if the gate is applied to a mixture that contained speech and the gate determines that the estimated audio does not contain speech, then it is already an indication that the extractor has failed to extract the target speaker, *(ii)* if the gate is applied to a mixture that does not contain speech and the gate determines that the estimated audio does contain speech, then it is already an indication that the extractor has hallucinated, *(iii)* if the gate is applied to a mixture that does not contain speech and the gate determines that the estimated audio does not contain speech, then it is already an indication that the extractor has correctly determined that there is no speech, and *(iv)* this method is a practical implementation in a real system thus has a natural application to the evaluation of the extractor. Therefore it covers all possible scenarios and is a practical solution to the problem of hallucination. This method is support by [22] and is applied for both the ASR and live judge.

4.1.3. Latency measures

During the course of experiments, the latency of the different systems was measured to compare the implemented model against *(i)* the latency specification for live transcription, and *(ii)* the latency of established systems. There are two main components to consider: time to consume and time to process the audio. The first component considers the real-time factor (RTF). The RTF is the ratio of the time taken to process the audio to the length of the audio, and for a model to be able to keep up with real-time audio, the RTF must be less than 1. This is The second component considers the latency created by the architecture itself which cannot be improved by faster hardware. To produce the output frame for a given window, the model must first have

received that entire window, and the final lookahead samples (if considered) of it have not yet arrived. This forces a delay of the processing of the window as per the subsection 3.2.1. For the model to be able to keep up with real-time audio, the RTF needs to be less than one, and secondly, the budget for processing a frame must be less than 200-300ms as a standard for live transcription. Finally, before either timing figure is interpreted, a probe checks whether a system can stream at all. A real-time factor cannot distinguish a model that is merely slow from one that requires the entire utterance before it can produce anything. The probe replaces all audio after a time T and measures how far the output *before* T moves: if changing the future changes what the model has already produced, then that output depends on audio which has not yet arrived, and the model cannot be streamed. For a strictly causal model the movement must be zero. The replacement is made at matched loudness so that the overall level of the input is preserved.

4.1.4. Model selection rule

At the end of a training pass, the model has one set of weights. The objective function however, has many components that can be minimised and thus the final best model is not necessarily the one with the overall validation loss at the lowest point.

The weight w, as found in equation 3.11, balancing the target-present and target-absent halves of the objective was calculated from the data split to match the proportion of target-absent crops in the set. This is done as a means to ensure that each branch contributes a proportionate amount to the total loss. During experiments, it was found that the absent branch could be continuously optimised further, but eventually would worsen the present branch. This is because the absent branch is a much simpler task to solve than the present branch, where the model can simply learn to output silence and thus the loss will continue to decrease which is not a desirable outcome.

Selection is therefore made on the *present branch alone*. The rationale behind this is that the training will introduce a loss term to encourage silence on absent cases, but it should not be the marker of a successful model. However, this does not mean to abandon the absent branch entirely. The absent branch ensures that a model will learn not to talk over silence, which on this data affects roughly a quarter of trials. Silence is therefore handled as an eligibility constraint. For a model to be considered for selection it must be quiet enough on target-absent crops to be considered and only if the condition is met will the model be selected. Therefore the model must be at least 10 dB down on the absent branch to be considered for selection.

4.1.5. Model comparison

[TODO] Compare to WeSep methodology

Chapter 5

Results

[TODO:] Highlight that the model still has capacity to learn more with more training data and more parameters. Just that we needed a proof of concept. Also need to maybe put CARTSE's results in here for reference on another dataset.

5.1. Content fidelity

5.1.1. The live judge

Table 5.1: Content recovered by the live judge. `gemini-3.7-flash`, audio in / text out

system	LCF-WER (%)	ICR@2 (%)	mean leak (%)	invented /trial	FR@2 (%)
Floor (do nothing)	63.27	75.73	63.33	1.24	32.0
Ours (4,976 trials)	56.72	62.14	50.15	1.83	41.7
Ours (9,955 trials)	55.59	61.17	48.32	2.01	43.7
Extension (gate)	—	—	—	—	—
WeSep	**26.40**	**25.24**	**12.92**	**1.80**	**38.8**
Ceiling (clean)	1.05	0.00	0.00	0.20	1.0

5.1.2. The offline transcriber

Table 5.2: Content recovered by the offline transcriber, `faster-whisper small.en`. Lower is better in every column.

system	WER (%)	ICR@2 (%)	mean leak (%)	invented /trial	FR@2 (%)
Floor (do nothing)	65.22	66.99	51.30	2.61	57.28
Ours (4,976 trials)	59.05	54.37	39.13	**2.73**	**60.61**
Ours (9,955 trials)	59.52	50.49	34.58	3.08	68.00
Extension (gate)	—	—	—	—	—
WeSep	**34.60**	**15.53**	**9.02**	3.03	73.79
Ceiling (clean)	5.85	0.00	0.00	0.88	19.42

5.1.3. Headroom captured

Table 5.3: Headroom captured: the fraction of the floor-to-ceiling gap each system closes, derived from Table 5.1 and Table 5.2. Higher is better.

system	listener	LCF-WER (%)	ICR@2 (%)	mean leak (%)
Ours (4,976)	judge	10.5	17.9	20.8
Ours (9,955)	judge	12.3	19.2	23.7
WeSep	judge	**59.3**	**66.7**	**79.6**
Ours (4,976)	ASR	10.4	18.8	23.7
Ours (9,955)	ASR	9.6	24.6	32.6
WeSep	ASR	**51.6**	**76.8**	**82.4**

5.2. Error composition

Table 5.4: Substitution (S), deletion (D) and insertion (I) rates for each listener. Lower is better.

	judge			offline ASR		
system	S	D	I	S	D	I
Floor	29.22	5.97	28.08	32.89	9.28	23.05
Ours (4,976)	27.07	3.43	26.22	28.06	12.60	18.39
Ours (9,955)	25.64	3.00	26.95	28.20	11.79	19.53
Extension (gate)	—	—	—	—	—	—
WeSep	10.62	6.78	8.99	18.51	8.91	7.19
Ceiling	0.87	0.15	0.03	3.43	0.67	1.75

5.3. Signal-domain separation

Table 5.5: Signal-domain decomposition, in dB; higher is better. The ceiling reaches $30\,\mathrm{dB}$ by construction, being the cap imposed by the denominator floor $\tau = 10^{-3}$, and is not a measurement.

system	SDR	SIR	SAR
Floor (do nothing)	-1.12	-1.12	$+30.00$
Ours (4,976 trials)	$+0.86$	$+3.21$	$+10.34$
Ours (9,955 trials)	$+1.29$	$+3.66$	$\mathbf{+11.11}$
Extension (gate)	—	—	—
WeSep	$\mathbf{+4.68}$	$\mathbf{+10.34}$	$+8.26$
Ceiling (clean)	$+30.00$	$+30.00$	$+30.00$

5.4. Perceptual quality

Table 5.6: DNSMOS, personalised model. All scores are between 1 and 5 where the higher the score, the better.

system	P808	SIG	BAK	OVRL
Floor (do nothing)	2.913	4.090	2.031	2.497
Ours (4,976 trials)	2.937	3.366	2.266	2.237
Ours (9,955 trials)	2.955	3.233	2.286	2.183
Extension (gate)	—	—	—	—
WeSep	**3.010**	**3.450**	**2.640**	**2.510**
Ceiling (clean)	3.550	4.175	3.592	3.429

5.5. Latency and throughput

Table 5.7: Latency and throughput, measured over 2,250 chunks of $80\,\mathrm{ms}$ on CPU (i5-1135G7, 4 threads). The requirement is RTF < 1 and latency within 200–$300\,\mathrm{ms}$.

	RTF		latency (ms)		meets
system	mean	p99	mean	p99	budget
Baseline	0.528	0.706	162.2	176.5	yes
Extension (gate)	—	—	—	—	—
WeSep	2.854	5.300	348.3	544.0	no[‡]

[‡] And cannot stream at all, independently of its throughput. See the causality result above.

Chapter 6

Conclusion

Bibliography

[1] Y. Luo and J. Yu, "Music source separation with Band-Split RNN," *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 31, pp. 1893–1901, 2023.

[2] J. Yu, H. Chen, Y. Luo, R. Gu, and C. Weng, "High fidelity speech enhancement with Band-split RNN," in *Proc. Interspeech 2023*, 2023, pp. 2483–2487.

[3] K. Zhang, J. Li, S. Wang, Y. Wei, Y. Wang, Y. Wang, and H. Li, "Multi-level speaker representation for target speaker extraction," in *Proc. IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2025.

[4] S. Wang, Z. Qian, K. Zhang, J. Han, Z. Liu, X. Yu, H. Li, M. Delcroix, K. Yu, L. Xie, M. Li, and H. Li, "SLT 2026 REAL-TSE challenge: Real-world target speaker extraction from conversational recordings," arXiv, Tech. Rep. arXiv:2607.15198, 2026.

[5] V. Panayotov, G. Chen, D. Povey, and S. Khudanpur, "Librispeech: An ASR corpus based on public domain audio books," in *Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. South Brisbane, QLD, Australia: IEEE, 2015, pp. 5206–5210, source of every mixture in this work and of the starred references $y^{\star}$. Chosen because it supplies both a clean per-speaker signal and exact ground-truth text, neither of which real conversational corpora provide.

[6] G. Wichern, J. Antognini, M. Flynn, L. R. Zhu, E. McQuinn, D. Crow, E. Manilow, and J. Le Roux, "WHAM!: Extending speech separation to noisy environments," in *Proceedings of Interspeech*, 2019, pp. 1368–1372, licensed CC BY-NC 4.0, which is the binding constraint on redistribution of every constructed mixture.

[7] R. Scheibler, E. Bezzam, and I. Dokmanić, "Pyroomacoustics: A python package for audio room simulation and array processing algorithms," in *Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2018, pp. 351–355, version 0.10.1. Shoebox room impulse responses by the image-source method, with a Sabine T60 feasibility check.

[8] L. Li and S. Seki, "CARTSE submission to the REAL-TSE challenge track 1: Online target speaker extraction," CyberAgent, Inc., Tech. Rep., 2026, system description.

[9] W. C. Sabine, *Collected Papers on Acoustics*. Cambridge, MA: Harvard University Press, 1922.

[10] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.

[11] Y. N. Dauphin, A. Fan, M. Auli, and D. Grangier, "Language modeling with gated convolutional networks," in *Proc. 34th International Conference on Machine Learning (ICML)*, ser. Proceedings of Machine Learning Research, vol. 70, 2017, pp. 933–941.

[12] Google DeepMind, "Gemini Live API," https://ai.google.dev/gemini-api/docs/live, 2026, accessed: 2026-08-31. Exact model identifier to be recorded with each result.

[13] SYSTRAN, "faster-whisper: Faster Whisper transcription with CTranslate2," https://github.com/SYSTRAN/faster-whisper, 2026, version 1.2.1. Accessed: 2026-08-31.

[14] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, "Robust speech recognition via large-scale weak supervision," in *Proceedings of the 40th International Conference on Machine Learning (ICML)*, ser. Proceedings of Machine Learning Research, vol. 202, 2023, pp. 28 492–28 518, also arXiv:2212.04356. The English text normaliser used in this work is defined in Appendix C.

[15] whisper-normalizer contributors, "whisper-normalizer: Standalone text normalisation from OpenAI Whisper," https://pypi.org/project/whisper-normalizer/, 2026, version 0.1.15. Implements the EnglishTextNormalizer of [14]. Accessed: 2026-08-31.

[16] S. Bird, E. Klein, and E. Loper, *Natural Language Processing with Python*. O'Reilly Media, 2009, the English stopword list is taken from the NLTK stopwords corpus and snapshotted into this project's source tree so the list cannot change with a library upgrade.

[17] International Telecommunication Union, "Subjective test methodology for evaluating speech communication systems that include noise suppression algorithm," ITU-T, Geneva, Recommendation P.835, Nov. 2003, rates the same clip three times on separate axes (SIG, BAK, OVRL) so a system cannot hide noise by damaging speech. Approved 11/2003 and not revised since.

[18] ——, "Subjective evaluation of speech quality with a crowdsourcing approach," ITU-T, Geneva, Recommendation P.808, Jun. 2018, one overall rating per clip. A separate protocol from P.835, not a summary of it. Revised 06/2021; the 2018 text is the one the DNSMOS papers follow.

[19] C. K. A. Reddy, V. Gopal, and R. Cutler, "DNSMOS P.835: A non-intrusive perceptual objective speech quality metric to evaluate noise suppressors," in *Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2022, pp. 886–890, returns SIG, BAK and OVRL separately, per ITU-T P.835. OVRL is the score the REAL-TSE Challenge saw over-optimised.

[20] ——, "DNSMOS: A non-intrusive perceptual objective speech quality metric to evaluate noise suppressors," in *Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2021, pp. 6493–6497.

[21] J. Le Roux, S. Wisdom, H. Erdogan, and J. R. Hershey, "SDR – half-baked or well done?" in *Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2019, pp. 626–630.

[22] "Investigation of Whisper ASR hallucinations induced by non-speech audio," arXiv:2501.11378, 2025, evaluates VAD gating against confidence thresholding and pattern matching; finds both give meaningful reductions and that no single method eliminates the problem.

[23] jiwer contributors, "jiwer: Evaluate automatic speech recognition," https://github.com/jitsi/jiwer, 2026, version 4.0.0. Accessed: 2026-08-31.

[24] International Telecommunication Union, "Algorithms to measure audio programme loudness and true-peak audio level," ITU-R, Geneva, Recommendation BS.1770-4, 2015, integrated loudness, as implemented by `pyloudnorm` 0.2.0. All trial levels are set in these units.

[25] Silero Team, "Silero VAD: Pre-trained enterprise-grade voice activity detector," https://github.com/snakers4/silero-vad, 2021, version 6.2.1, used unmodified. Defines what counts as speech for both the corpus index and the evaluation speech gate.

Appendix A

Evaluation protocol details

A.1. Prompt for live-model transcription

```
Write down the words spoken in this audio, exactly as spoken.

If no words are audible, leave the transcript empty and set status to no_speech.
```

A.2. Pinned components of the measuring instrument

Component	Identity
Text normaliser	`whisper-normalizer` 0.1.15, `EnglishTextNormalizer` [14, 15]
Baseline transcriber	`faster-whisper` 1.2.1, `small.en`, int8 quantisation, greedy decoding [13, 14]
Word error rate	`jiwer` 4.0.0 [23]
Stopword list	`NLTK` English stopwords [16], snapshotted into the source tree so the list cannot change with a library upgrade
Judge model	*to be recorded*
Random seed	42

Table A.1: Components of the evaluation instrument, pinned by version. A change to any row invalidates comparison with numbers reported before it.

Appendix B

Room construction

Each trial is rendered in its own simulated room rather than convolved with a recorded impulse response, so that the acoustics are a controllable variable rather than a property of whichever room happened to be recorded. Rooms are rectangular ("shoebox") and the impulse responses are generated by the image-source method using `pyroomacoustics` 0.10.1 [7].

For every trial the renderer draws the room's three dimensions, a reverberation time, and independent positions for the microphone and the two talkers. Wall absorption is not drawn directly: it is solved for from the requested reverberation time and the room's surface area by the Sabine relation, and the draw is rejected and retaken if the combination is not physically realisable. Both talkers are placed in the same room and share the microphone, so the two speech signals arrive with different but consistent room responses.

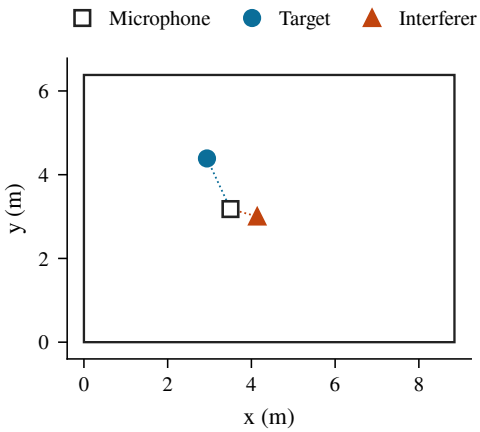

Figure B.1: Plan view of the room rendered for trial `train-42-007495`. The room measures $8.85 \times 6.38 \times 3.14\,\mathrm{m}$ ($177\,\mathrm{m}^3$) with a reverberation time of $0.250\,\mathrm{s}$, for which the Sabine relation gives a wall absorption of 0.5479. The microphone sits at $(3.50, 3.18, 1.28)$ m, the target at $(2.94, 4.39, 1.66)$ m and the interferer at $(4.14, 3.00, 1.57)$ m, placing them $1.39\,\mathrm{m}$ and $0.72\,\mathrm{m}$ from the microphone respectively.

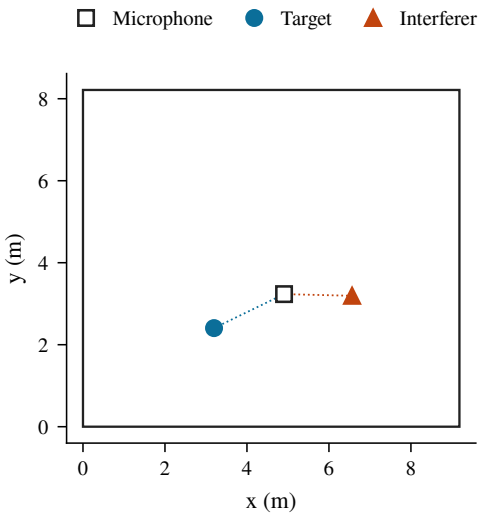

Figure B.2: Plan view of the room rendered for trial `train-42-004785`, the most reverberant in the split. The room measures $9.18 \times 8.21 \times 3.48\,\mathrm{m}$ ($263\,\mathrm{m}^3$) with a reverberation time of $0.600\,\mathrm{s}$, for which the Sabine relation gives a wall absorption of 0.2594 which is less than half that of Figure B.1, in a room $49\,\%$ larger. The microphone sits at $(4.91, 3.23, 1.49)$ m, the target at $(3.20, 2.40, 1.52)$ m and the interferer at $(6.56, 3.19, 1.64)$ m, placing them $1.90\,\mathrm{m}$ and $1.67\,\mathrm{m}$ from the microphone respectively.

Appendix C

Construction parameters

Every parameter below is drawn independently per trial from the stated distribution, except where a regime overrides it. Ranges written $[a, b]$ are sampled uniformly on that interval.

Parameter	Distribution	Meaning
Trial layout		
Mixture length	$[15.0, 20.0]$ s	Duration of the rendered trial
Overlap ratio	$[0.0, 0.7]$	Fraction of target speech with the interferer also talking
Overlap tolerance	0.03	Accepted deviation from the requested overlap
Target activity ratio	$[0.15, 0.78]$	Fraction of the trial in which the target speaks
Activity tolerance	0.1	Accepted deviation from the requested activity
Utterances per source	≤ 6	Cap on how many utterances one recording may contribute
Levels		
SIR	$[-5.0, 15.0]$ dB	Target loudness relative to the interferer
SNR	$[0.0, 20.0]$ dB	Target loudness relative to the noise bed
Target loudness	$[-33.0, -25.0]$ LUFS	Absolute level of the target, BS.1770 integrated [24]
Clip ceiling	0.95	Peak after which the whole trial is rescaled together
Room		
T_{60}	$[0.25, 0.6]$ s	Reverberation time
Room length, width	$[5.0, 10.0]$ m	Floor plan
Room height	$[3.0, 4.0]$ m	
Source distance	$[0.1, 2.0]$ m	Talker to microphone
Source height	$[0.9, 1.8]$ m	Talker, seated to standing
Microphone height	$[0.9, 1.8]$ m	
Early reflections	50 ms	Cutoff separating early echoes from the late tail
Enrolment		
Enrolment length	5.0 s	Single clip, levelled, carrying no room
EQ augmentation	$p = 0.5$	Three peaking biquads, log-uniform 120 Hz to 6400 Hz, ± 6 dB, $Q = 1.0$, RMS restored
Speaker pairing		
Same-gender fraction	0.5	Proportion of trials whose two talkers share a gender
Speech detection [25]		
Silence threshold	250 ms	Pause length after which a talker counts as stopped; **this defines overlap**
Speech probability	0.5	Frame-level threshold
Minimum speech	100 ms	Shortest run counted as speech
Speech padding	30 ms	Added either side of a detected run
Noise-bed rejection	0.5 s	Longest run of detected speech tolerated in a WHAM! clip
Regimes		
Regime weights	0.6 / 0.4	Proportion of trials drawn as `base` / `hard`
`base` SIR	$[0.0, 12.0]$ dB	Regime override
`base` SNR	$[8.0, 20.0]$ dB	Regime override
`base` activity	$[0.45, 0.78]$	Regime override
`base` T_{60}	$[0.25, 0.5]$ s	Regime override

Table C.1: Construction parameters and their sampling distributions.